

Module 0

Installation and Setup

Learning objectives

- Tool Installation: Automated and manual installation methods for all verification tools
- Environment Setup: Configure your development environment for efficient verification work
- First Testbenches: Create and run your first "Hello World" testbenches in both paradigms
- Waveform Analysis: Generate and view waveforms using GTKWave
- Project Structure: Organize your verification project following best practices
- Tool Comparison: Understand when to use iverilog vs Verilator

Prerequisites

- Have a Linux, macOS, or Windows (WSL2) system
- Have basic command-line knowledge
- Have Git installed (for cloning the repository)
- Have a C/C++ compiler available (GCC, Clang, or MSVC)

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["System Requirements and Prer"]

T2["Icarus Verilog (iverilog) In"]

T1 --> T2

T3["Verilator Installation"]

T2 --> T3

Set up verification environment with iverilog and Verilator

Overview

- This module covers the complete setup of your verification environment, including installation of i...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph tools["Toolchain"]

IV[iverilog] --> VVP[vvp]

VER[verilator] --> OBJ[obj_dir C++ model]

end

subgraph dut0["module0/dut"]

Module 0: DUT / block hierarchy (see module0/examples/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

INSTALL[install scripts] --> SMOKE[hello-world tests]

SMOKE --> WAVE[VCD + GTKWave]

Module 0: UVM layers, agents, and checkers for this phase.

1. Verification toolchain layout

- Host environment: Linux, macOS, or WSL2 with GCC/Clang, Make, and Git
- Icarus path: ``iverilog`` compiles RTL + TB → ``simv`` (or named binary) executed by ``vvp``
- Verilator path: ``verilator --cc`` emits C++ model under ``obj_dir/`` linked with a C++ testbench
- Waveform path: Simulators write VCD/FST; GTKWave visualizes signal transitions
- Course layout: ``scripts/install_*.sh``,
``module0/examples/``, ``module0/dut/``,
``module0/tests/``

2. Reference DUT hierarchy (Module 0)

- ``module0/dut/simple_gates/``: Combinational
 ``and_gate``, ``or_gate`` — minimal port lists (a, b, y)
- ``module0/dut/counters/``: ``simple_counter`` — clocked sequential logic with enable and reset
- Hierarchy depth: Flat RTL (no sub-modules) so install smoke tests stay fast
- TB attachment: Testbench modules instantiate DUT at top level; no bus fabric yet

3. First simulation artifact flow

- iverilog: ``iverilog -o sim tb.v dut.v`` → ``vvp sim`` →
optional ``$dumpfile` VCD`
- Verilator: ``verilator --cc --exe`` → ``make -C``
`obj_dir`` → ``./obj_dir/Vdut``
- Build dirs: Per-example ``Makefile`` targets;
artifacts under ``build/`` or ``obj_dir/`` (gitignored)

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

A[verify tools] --> B[smoke DUT]

B --> C[compare iverilog vs Verilator]

Module 0: how tests, checks, and metrics close verification goals.

1. Install and environment verification

- Version checks: ``iverilog -v``, ``verilator --version``, ``gtkwave --version``
- PATH and compiler: Confirm ``g++``/``gcc`` available for Verilator link step
- Self-check: ``./scripts/module0.sh --check`` validates examples and DUT compiles

2. Hello-world test methodology

- Smoke test: Minimal compile-and-run per tool (no functional depth required)
- Pass criteria: Clean compile, simulation exits 0, expected ``$display`` or C++ ``printf``
- Waveform spot-check: Open generated VCD in GTKWave to confirm clock/reset toggles

3. Debugging and regression basics

- Log-first: Capture simulator stdout; use ``-g` /
`--trace`` when examples enable it
- Compare tools: Run same DUT with iverilog vs Verilator to see flow differences
- Repeatability: Document working directory and Make target in ``commands_to_try.txt``

Syllabus topics

1. System Requirements and Prerequisites (1/3)

- Operating System Support
- Linux (Ubuntu/Debian, CentOS/RHEL, Fedora)
- macOS (Intel and Apple Silicon)
- Windows (WSL2 recommended)
- Hardware Requirements
- Minimum 4GB RAM (8GB+ recommended)

1. System Requirements and Prerequisites (2/3)

- 5GB free disk space
- Multi-core processor recommended
- Software Prerequisites
 - C/C++ compiler (GCC 7+, Clang 10+, or MSVC)
 - Make or Ninja build system
 - Git

1. System Requirements and Prerequisites (3/3)

- Perl (for some tools)

2. Icarus Verilog (iverilog) Installation (1/5)

- What is iverilog?
- Open-source Verilog/SystemVerilog simulator
- Good for learning and prototyping
- Supports most Verilog constructs
- Limited SystemVerilog support
- Automated Installation (Recommended)

2. Icarus Verilog (iverilog) Installation (2/5)

- Using the installation script:
- The script automatically:
- Checks for existing installations
- Installs system dependencies (build tools, libraries)
- Builds and installs iverilog
- Verifies the installation

2. Icarus Verilog (iverilog) Installation (3/5)

- Manual Installation Methods
- Linux Installation
- Ubuntu/Debian: ``sudo apt-get install iverilog gtkwave``
- CentOS/RHEL: ``sudo yum install iverilog gtkwave``
- Fedora: ``sudo dnf install iverilog gtkwave``
- Building from source (latest features)

2. Icarus Verilog (iverilog) Installation (4/5)

- macOS Installation
- Homebrew installation: ``brew install icarus-verilog gtkwave``
- Building from source
- Windows/WSL2 Installation
- Installing in WSL2 Ubuntu: ``sudo apt-get install iverilog gtkwave``
- Building from source in WSL2

2. Icarus Verilog (iverilog) Installation (5/5)

- Verification Steps
- Check iverilog version: ``iverilog -v``
- Check vvp runtime: ``vvp -v``
- Run simple test: ``echo 'module test; initial $display("Hello"); endmodule' | iverilog -o test -&&...``

3. Verilator Installation (1/5)

- What is Verilator?
- Fast Verilog/SystemVerilog simulator
- Generates C++ code from Verilog
- Excellent performance
- Limited SystemVerilog support (some features)
- Automated Installation (Recommended)

3. Verilator Installation (2/5)

- Using the installation script:
- The script automatically:
- Checks for existing installations
- Installs system dependencies (build tools, libraries)
- Sets up git submodule in `tools/verilator/`
- Builds and installs Verilator

3. Verilator Installation (3/5)

- Verifies the installation
- Manual Installation Methods
- Linux Installation
- Ubuntu/Debian: ``sudo apt-get install verilator``
- CentOS/RHEL: ``sudo yum install verilator``
- Fedora: ``sudo dnf install verilator``

3. Verilator Installation (4/5)

- Building from source (latest features)
- macOS Installation
- Homebrew installation: ``brew install verilator``
- Building from source
- Windows/WSL2 Installation
- Installing in WSL2 Ubuntu: ``sudo apt-get install verilator``

3. Verilator Installation (5/5)

- Building from source in WSL2
- Verification Steps
- Check Verilator version: ``verilator --version``
- Run simple test: Create a minimal Verilog file and compile with Verilator

4. GTKWave Installation (1/2)

- What is GTKWave?
- Waveform viewer for VCD and FST files
- Essential for debugging testbenches
- Open-source and cross-platform
- Installation
- Usually installed with iverilog package

4. GTKWave Installation (2/2)

- Linux: ``sudo apt-get install gtkwave`` (or included with iverilog)
- macOS: ``brew install gtkwave`` (or included with icarus-verilog)
- Windows: Available as standalone installer
- Verification Steps
- Check GTKWave: ``gtkwave --version``
- Open a VCD file: ``gtkwave waveform.vcd``

5. Build System Configuration (1/4)

- Makefile Setup
- Basic Makefile for iverilog
- Basic Makefile for Verilator
- Compilation flags
- Include paths
- Test execution

5. Build System Configuration (2/4)

- iverilog Compilation
- Compilation command: ``iverilog -o output input.v``
- Include paths: ``-I<directory>``
- Define macros: ``-D<macro>``
- Timescale handling
- Multi-file compilation

5. Build System Configuration (3/4)

- Verilator Compilation
- Compilation command: ``verilator --cc --exe design.v testbench.cpp``
- SystemVerilog flags: ``-sv``, ``--timing``
- Include paths: ``-I<directory>``
- Optimization flags: ``-O0``, ``-O1``, ``-O2``, ``-O3``
- Coverage options: ``--coverage``

5. Build System Configuration (4/4)

- Waveform generation: `--trace`, `--trace-fst`

6. IDE Setup and Configuration (1/3)

- Recommended IDEs
- VS Code with Verilog/SystemVerilog extension
- Verible (SystemVerilog linter/formatter)
- Vim/Neovim with LSP
- Emacs with Verilog mode
- VS Code Configuration

6. IDE Setup and Configuration (2/3)

- SystemVerilog extension setup
- iverilog integration
- Verilator integration
- Debugging configuration
- Task runner setup for simulations
- Extension recommendations

6. IDE Setup and Configuration (3/3)

- Editor Configuration
- SystemVerilog formatting (Verible)
- Linting (Verilator, Verible)
- Code formatting on save
- Syntax highlighting

7. Project Structure Setup (1/4)

- Directory Structure
- Source code organization
- Test directory structure
- DUT (Design Under Test) organization
- Configuration files
- ``tools/`` directory for tools (Verilator as git submodule)

7. Project Structure Setup (2/4)

- Git Submodules Management
- Verilator is managed as a git submodule in
`tools/verilator/`
- Initialize submodules:
`./scripts/init\_submodules.sh` or `git submodule
update --init --recursive`
- Update submodules: `./scripts/update\_submodules.sh`
or `git submodule update --remote`
- Add new submodule: `./scripts/add_submodule.sh
<repo_url> <path>`
- Remove submodule: `./scripts/remove_submodule.sh
<path>`

7. Project Structure Setup (3/4)

- Makefile/Configuration
- Simple Makefile for running tests
- iverilog configuration
- Verilator configuration
- Environment variable management
- Version Control

7. Project Structure Setup (4/4)

- Git initialization
- .gitignore for Verilog and simulation (include build artifacts, waveforms)
- Initial commit structure
- Git submodules in `.gitmodules` file

8. First "Hello World" Verification Test (1/8)

- Prerequisites
- Ensure all tools are installed
- Verify tools are accessible: ``iverilog -v`` and ``verilator --version``
- iverilog Hello World
- Location:
``module0/examples/iverilog_basics/hello_world.v``
- Purpose: Simplest possible testbench to verify iverilog installation

8. First "Hello World" Verification Test (2/8)

- Key Concepts:
- ``module`` declaration (testbench has no ports)
- ``initial`` block for procedural code execution
- ``$display`` system task for output
- ``$finish`` system task to end simulation
- Time delays with ``#delay``

8. First "Hello World" Verification Test (3/8)

- Compilation: ``iverilog -o hello_world hello_world.v``
- Simulation: ``vvp hello_world``
- Understanding Output: Console messages confirm successful execution
- GTKWave Waveform Example
- Location:
``module0/examples/iverilog_basics/gtkwave_example.v``
- Purpose: Comprehensive example demonstrating waveform generation and analysis

8. First "Hello World" Verification Test (4/8)

- Key Concepts:
- VCD (Value Change Dump) file generation with
 `\$dumpfile` and `\$dumpvars`
- Clock generation using `forever` loops
- Multiple signal types (clock, data, control signals)
- Signal transitions for timing analysis
- Test phases and patterns

8. First "Hello World" Verification Test (5/8)

- Compilation: ``iverilog -o gtkwave_example
gtkwave_example.v ../../dut/simple_gates/and_gate.v``
- Simulation: ``vvp gtkwave_example`` (generates
``gtkwave_example.vcd``)
- Viewing: ``gtkwave gtkwave_example.vcd`` or ``make
view_waveforms``
- GTKWave Features Demonstrated:
- Signal hierarchy navigation
- Time marker placement

8. First "Hello World" Verification Test (6/8)

- Zoom and pan operations
- Save file creation for quick reload
- Verilator Hello World
- Location:
 - ``module0/examples/verilator_basics/hello_world.cpp``
 - and ``hello_world.v``
- Purpose: Simplest possible C++ testbench to verify Verilator installation
- Key Concepts:

8. First "Hello World" Verification Test (7/8)

- Verilator compilation process (``verilator --cc --exe --build``)
- Generated C++ class structure (``V<module_name>``)
- DUT instantiation in C++
- ``eval()`` method for simulation
- ``final()`` method for cleanup
- Verilator initialization with ``Verilated::commandArgs()``

8. First "Hello World" Verification Test (8/8)

- Compilation: ``verilator --cc --exe --build
hello_world.v hello_world.cpp``
- Running: ``./obj_dir/Vhello_world``
- Understanding Output: Console messages confirm successful execution

9. Tool Comparison and Selection Criteria (1/4)

- iverilog Characteristics
- Verilog/SystemVerilog testbenches
- Good for learning
- Easy to use
- Good waveform support
- Moderate performance

9. Tool Comparison and Selection Criteria (2/4)

- Verilator Characteristics
- C++ testbenches
- Excellent performance
- Good for large designs
- Requires C++ knowledge
- Limited SystemVerilog support

9. Tool Comparison and Selection Criteria (3/4)

- When to Use Each
- Use iverilog for:
 - Learning Verilog testbench concepts
 - Quick prototyping
 - Simple to medium complexity designs
- When you prefer Verilog syntax

9. Tool Comparison and Selection Criteria (4/4)

- Use Verilator for:
- High-performance simulation
- Large designs
- Integration with C++ libraries
- Complex testbenches with advanced data structures

10. Common Pitfalls and Solutions

- See docs/MODULE0.md

11. Troubleshooting Common Issues (1/8)

- Cause: iverilog not installed or not in PATH
- Solution:
- Verify: ``iverilog -v`` should show version
- Cause: SystemVerilog syntax not supported by default
- Solution: Use ``-g2012`` flag for SystemVerilog support
- Note: Some SystemVerilog features may still be limited

11. Troubleshooting Common Issues (2/8)

- Cause: Missing include path
- Solution: Add include directory with ``-I`` flag
- Cause: VVP runtime not installed or not in PATH
- Solution: VVP is usually installed with iverilog.

Reinstall if missing:

- Cause: Verilator not installed or not in PATH
- Solution:

11. Troubleshooting Common Issues (3/8)

- Cause: Installed version is too old
- Solution: Install from source for latest version
- Cause: C++ compiler not installed
- Solution: Install build essentials
- Cause: Verilator headers not found
- Solution: Check Verilator installation and include paths

11. Troubleshooting Common Issues (4/8)

- Cause: Missing Verilator library linking
- Solution: Ensure Makefile includes Verilator libraries
- Cause: GTKWave not installed
- Solution:
- Cause: File format issue or corrupted file
- Solution:

11. Troubleshooting Common Issues (5/8)

- Verify VCD file was generated correctly
- Check file permissions
- Try regenerating: `vvp executable` (should create .vcd file)
- Issue: Package manager version may be outdated
- Solution: Use installation scripts or build from source
- Issue: Homebrew version may be outdated

11. Troubleshooting Common Issues (6/8)

- Solution: Use `--source` flag to build from source
- Issue: Some tools may not work in Windows directly
- Solution: Use WSL2 Ubuntu and install there
- Issue: `make: \*\*\* No targets specified and no makefile found`
- Solution: Ensure you're in the correct directory with a Makefile
- Issue: Files not found during compilation

11. Troubleshooting Common Issues (7/8)

- Solution: Check relative paths and use ``-I`` flags correctly
- VS Code: Install "SystemVerilog" extension by mshr-h
- Vim: Install vim-systemverilog plugin
- Verification: Open a `.sv`` file and check for syntax highlighting
- Issue: IDE can't find included files
- Solution: Configure include paths in IDE settings

11. Troubleshooting Common Issues (8/8)

- VS Code: Add to `settings.json`:
- Issue: Can't set breakpoints or debug
- Solution:
- For Verilator: Use GDB with compiled executable
- For iverilog: Use VCD waveforms for debugging

12. Verification Checklist (1/3)

- C/C++ compiler installed and working
- iverilog installed and verified
- Using script: ``./scripts/install_iverilog.sh``
- Verify: ``iverilog -v`` and ``vvp -v``
- Verilator installed and verified
- Using script: ``./scripts/install_verilator.sh``

12. Verification Checklist (2/3)

- Verify: ``verilator --version``
- GTKWave installed and verified
- Verify: ``gtkwave --version``
- All tools verified together
- IDE configured and working
- First test runs successfully (both iverilog and Verilator)

12. Verification Checklist (3/3)

- Can create and run simple testbench
- Understand basic project structure
- Know how to get help when stuck
- Understand tool differences and when to use each

Hands-on examples

Module 0 self-check

```
$ ./scripts/module0.sh --check
```

Terminal: module0_check

```
=====
Module 0: environment self-check
=====
[OK] module0/ present
[OK] module0/examples/ present
[OK] iverilog: Icarus Verilog version 12.0 (stable) ()
[INFO] verilator not installed (needed for Module 2+ labs: ./scripts/install_verilator.sh)
All required checks passed
```

Exercise scaffold

```
./scripts/module0.sh --scaffold
```

Demo: iverilog basics

```
$ cd module0/examples/iverilog_basics && make hello_world
```

Terminal: ex01_iverilog_basics

Compiling hello_world...

Compile Verilog file into executable

-o: output filename

iverilog -o hello_world hello_world.v

Running hello_world...

Run compiled simulation

vvp hello_world

=====

Hello World from Icarus Verilog!

=====

If you see this message, iverilog is working correctly.

Time: 0

=====

hello_world.v:66: \$finish called at 10 (1s)

Demo: Verilator basics

```
$ ls -la module0/examples/verilator_basics && head -20  
module0/examples/verilator_basics/Makefile
```

Terminal: ex02_verilator_basics

```
total 20  
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 .  
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 ..  
-rwxrwxrwx 1 yongfu yongfu 2786 May 28 12:19 Makefile  
-rwxrwxrwx 1 yongfu yongfu 5223 May 28 12:19 and_gate_test.cpp  
-rwxrwxrwx 1 yongfu yongfu 4122 May 28 12:19 hello_world.cpp  
-rwxrwxrwx 1 yongfu yongfu 213 May 28 12:19 hello_world.v  
# Makefile for Verilator examples  
#  
# This Makefile automates compilation and simulation of Verilator testbenches.  
#  
# Usage:  
#   make [target]      # Build and run specific example  
#   make all           # Build and run all examples  
#   make clean         # Remove build artifacts  
#  
# Prerequisites:  
# - Verilator installed (see scripts/install_verilator.sh)  
# - C++ compiler (g++ or clang++)  
# - Make build system  
#  
# Build Process:  
# 1. Verilator generates C++ code from Verilog  
# 2. C++ compiler compiles generated code + testbench  
# 3. Linker creates executable in obj_dir/  
# 4. Run executable to simulate  
#
```


Demo: Tool comparison

```
$ cd module0/examples/comparison && ls -la && head -30 README.md  
2>/dev/null || head -30 Makefile
```

Terminal: ex03_comparison

```
total 4  
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 .  
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 ..  
-rwxrwxrwx 1 yongfu yongfu 1445 May 28 12:19 README.md  
# Comparison: iverilog vs Verilator
```

This directory contains side-by-side comparisons of testbenches written for iverilog (Verilog) a

Key Differences

iverilog (Verilog Testbenches)

- Written in Verilog/SystemVerilog
- Uses `\$display`, `\$monitor`, `\$strobe` for output
- Uses `#` delays for timing
- Uses `initial` and `always` blocks
- Generates VCD files with `\$dumpfile` and `\$dumpvars`
- Compiled with `iverilog`, run with `vvp`

Verilator (C++ Testbenches)

- Written in C++
- Uses `std::cout` for output
- Uses simulation loop for timing
- Uses C++ control structures (loops, conditionals)
- Generates VCD files with Verilator tracing API
- Compiled with `verilator`, linked with C++ compiler

Example Comparisons

See the individual example directories for detailed comparisons:

- `and\_gate/` - AND gate testbench comparison
- `counter/` - Counter testbench comparison (coming soon)

When to Use Each

Practice & assessment

Exercises

- Installation Verification
- Environment Setup
- First Tests
- IDE Configuration
- Project Structure

Exercises

- Tool Comparison

Assessment checklist

- Can install all required tools independently
- Can set up iverilog
- Can verify iverilog installation
- Can set up Verilator
- Can verify Verilator installation
- Can install and verify GTKWave

Summary & next steps

- Set up verification environment with iverilog and Verilator
- Complete module0/CHECKLIST.md
- Run `./scripts/module0.sh --check`
- Next: Next module in course